

Lab 4 - Funky Muzak

Equipment / Materials:

- MPLAB X IDE + XC8 / pic-as
- Programmer / debugger (Pickit 3)
- Oscilloscope
- Frequency counter
- Breadboard + jumpers
- 4x4 Keypad
- Dip¹⁶ DIP switch bank
- Speaker + Class D driver circuit
- Lab book

Core Objectives: by the end of this lab,

- 1) Generate an exact 1kHz square wave on a pin using software timing.
- 2) Build and use a loop-¹⁶up table (LUT) to convert an input "note selection" into a delay value.
- 3) Maintain a stable output frequency while polling DIP switches.
- 4) Maintain a stable output frequency while scanning a 4x4 Keypad.
- 5) Verify timing with an oscilloscope and explain differences between "calculated" and "measured".

Background Notes: this lab is about timing discipline.

- Generate a stable square wave at a target frequency
 - Use a loop¹⁶ look-up table of delay values to produce different notes.
 - Keep your output pin toggling at the correct rate while polling inputs.
 - Integrate a 4x4 Keypad scan without wrecking your waveform
- In other words: you will learn how software timing falls apart the moment you add "one more thing"

- Instruction cycle timing: for midrange PIC's, the instruction cycle is typically

$$T_{cy} = 4 / F_{OSC}$$

- Delay routines must be based on instruction cycles because oscillator frequency may vary.

"Class D amplifier" notes: using an external driver/amplifier because:

- A PIC GPIO pin isn't a speaker driver.
- The driver IC + MOSFET stage provides current drive and switching power behavior
- The series capacitor blocks DC to the speaker
- The microcontroller pin provides a logic-level square wave only

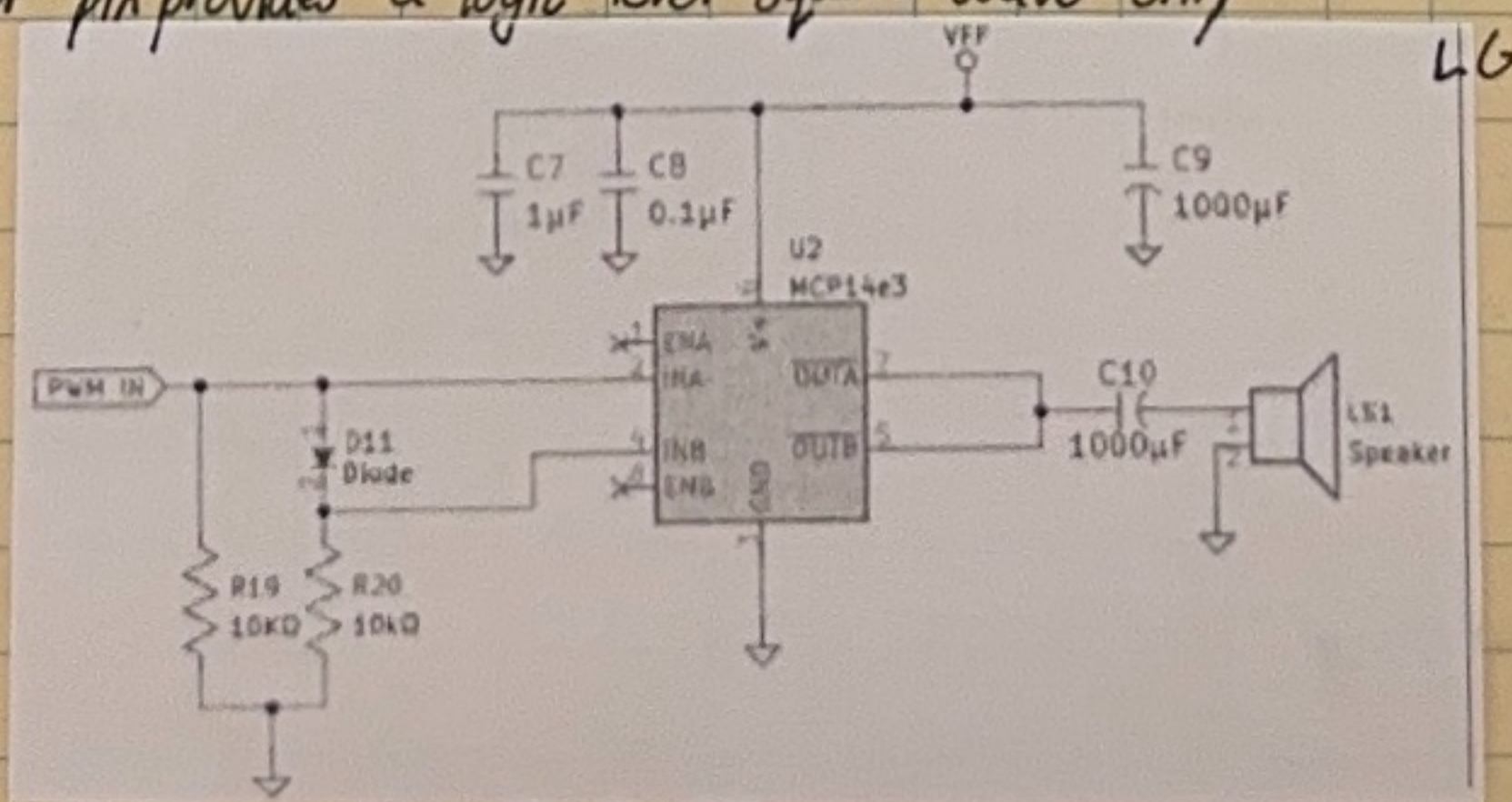


Figure 1: Class D Amplifier

• Part 1 - 1kHz Tone (Button Gated)

• Goal: Prove and generate a clean 1kHz square wave at the amplifier input.

• Part 1 Objectives:

- 1) Wire the amplifier input node (PWM IN) to RAO (or the lab-standard output pin).
- 2) Select a button input pin (recommended: R80, active-low with pull up).
- 3) Output a 1kHz square wave ($\approx 50\%$ duty) only while the button is pressed.
- 4) Capture the waveform and measure frequency/period.

• Required Behavior:

- Button not pressed - output pin forced low (silence)
- Button pressed - output pin toggles continuously at 1kHz
- Code must poll the button while running (do not "freeze" the program in a giant delay that ignores input for seconds)

• Calculations:

• Show:

- $F_{OSC} = 1\text{kHz}$
 - $T_{CY} = 500\mu\text{s}$
 - Half-period target for 1kHz: $0.5\text{ms} = PW/PS$
 - Show the cycle math for your delay routine (or calibrated loop structure).
 - See Pg 47 for equation and code to show the math.
- $$PW = 500\mu\text{s} \rightarrow \text{Duty Cycle \%} = 50\%$$
- $$PS = 500\mu\text{s}$$
- $$\text{Period} = 1\text{ms} = \text{frequency}/\text{kHz}$$

• Measurements:

- Oscilloscope capture of RAO (or PWM IN) showing:
 - Frequency near 1.000kHz
 - 50% duty cycle
 - At least one full cycle with readable scale
- Record measured frequency and compare to your calculated expectation.

• Evidence:

- circuit schematic and Class D interface • loading/electrical analysis • complete flowchart set
- final source code, fully included or referenced and easily accessible • nominal timing calculations
- expected timing using the previously measured instruction cycle time • scope capture & measurements
- frequency counter measurement • comparison of predicted and measured behavior
- Verification that no active buttons forces the output LOW • trouble shooting/change notes.

• In the Lab: • the tone output is logic low when button is not pressed

- the button status is read continuously • the active/inactive button condition is stored in a general register
- the same program path repeatedly toggles the output and delays one half cycle while the button remains active • the delay is implemented as reusable logic rather than embedded repeatedly in the main loop.

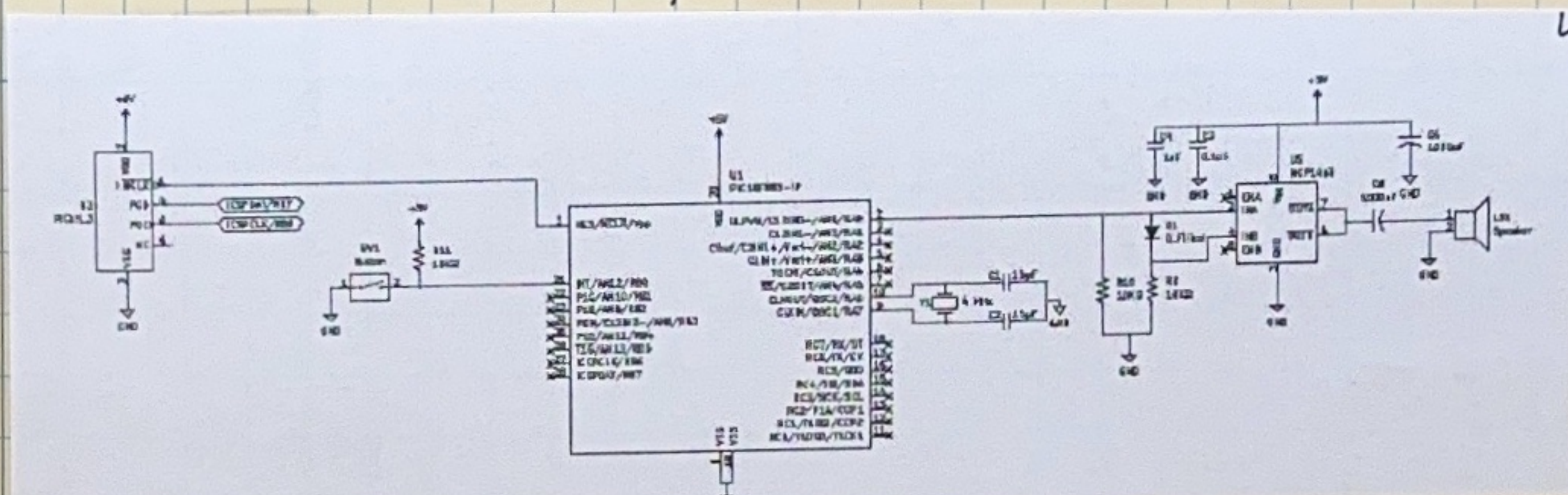


Figure 2: PIC16F883 with Class D Circuit

```

Main:
    BTFSC 0x06,0    ; Test RBO Skip the next line if button is pressed } 3Tcy
    GOTO ButtonUnpressed 2Tcy

ButtonPressed:
    MOVLW 0x01      ; 1Tcy
    XORWF 0x05,1    ; Toggle RA0
    CALL Delay      ; 2Tcy call to delay to cause a delay } 6Tcy
    GOTO Main       ; 2Tcy

ButtonUnpressed:
    BCF 0x05,0      ; 1Tcy Force RA0 low - silence } 3Tcy
    GOTO Main       ; 2Tcy

Delay:
    MOVLW 0xA3      ; 1Tcy move W 163 to count
    MOVWF count     ; 1Tcy make count 163 } 2Tcy

DelayLoop:
    DECFSZ count,1  ; 1Tcy if not zero, 2Tcy if zero
    GOTO DelayLoop  ; 2Tcy runs 163 times before skipping
    Return          ; 2Tcy return to call

End
    
```

only happens when button unpressed
27Tcy

$$N=163$$

$$\{((2+1)N)-1\} + 2$$

$$\{((2+1)163)-1\} + 2 = 490 \text{ } 492 \text{ Tcy}$$

$$492 \text{ Tcy} + 9 \text{ Tcy} = 501 \text{ Tcy}$$

Figure 3: Main code to produce a signal tone of 1KHz

Electrical Specifications: Unless otherwise indicated, $T_A = +25^\circ\text{C}$, with $4.5\text{V} \leq V_{DD} \leq 18\text{V}$.

Parameters	Sym	Min	Typ	Max	Units	Conditions
Input						
Logic '1', High Input Voltage	V_{IH}	2.4	1.5	—	V	
Logic '0', Low Input Voltage	V_{IL}	—	1.3	0.8	V	
Input Current	I_{IH}	-1	—	1	μA	$0\text{V} \leq V_{IH} \leq V_{DD}$
Input Voltage	V_{IN}	-5	—	$V_{DD}+0.3$	V	
Output						
High Output Voltage	V_{OH}	$V_{DD}-0.025$	—	—	V	DC Test
Low Output Voltage	V_{OL}	—	—	0.025	V	DC Test
Output Resistance, High	R_{OH}	—	2.5	3.5	Ω	$I_{OUT} = 10\text{mA}$, $V_{DD} = 18\text{V}$
Output Resistance, Low	R_{OL}	—	2.5	3.0	Ω	$I_{OUT} = 10\text{mA}$, $V_{DD} = 18\text{V}$
Peak Output Current	I_{PK}	—	4.0	—	A	$V_{DD} = 18\text{V}$ (Note 2)
Latch-Up Protection Withstand Reverse Current	I_{REV}	—	>1.5	—	A	Duty cycle $\leq 2\%$, $t \leq 300\mu\text{s}$

Table 1: D-Class Amp specs MCP14E3

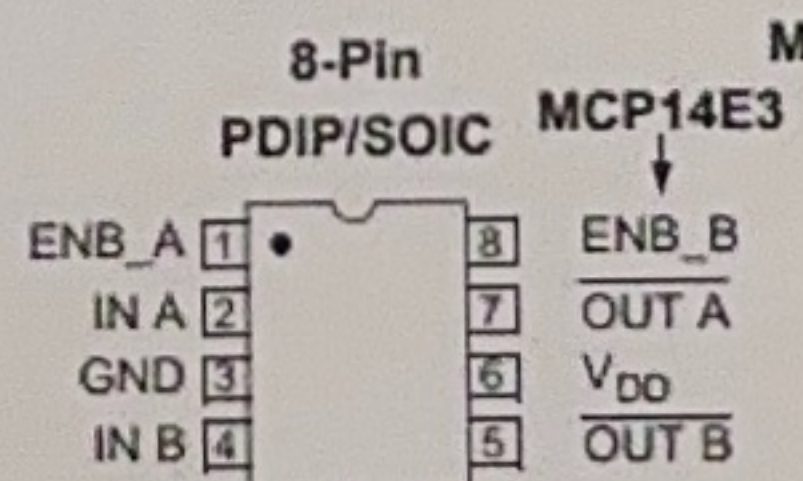


Figure 4: MCP14E3 pinout

The Class D Amp components were given but based on resistance values the current on input is around 1mA on the output the DC current can reach a max of 1.25A. = 5/4.0 speaker
The PIC won't even give more than a few μA of current

The next page shows the measurements for the signal into D-Class Amp and the speakers wave form.

Between L_2 (1000pF)
& Speaker

PORTA
Signal

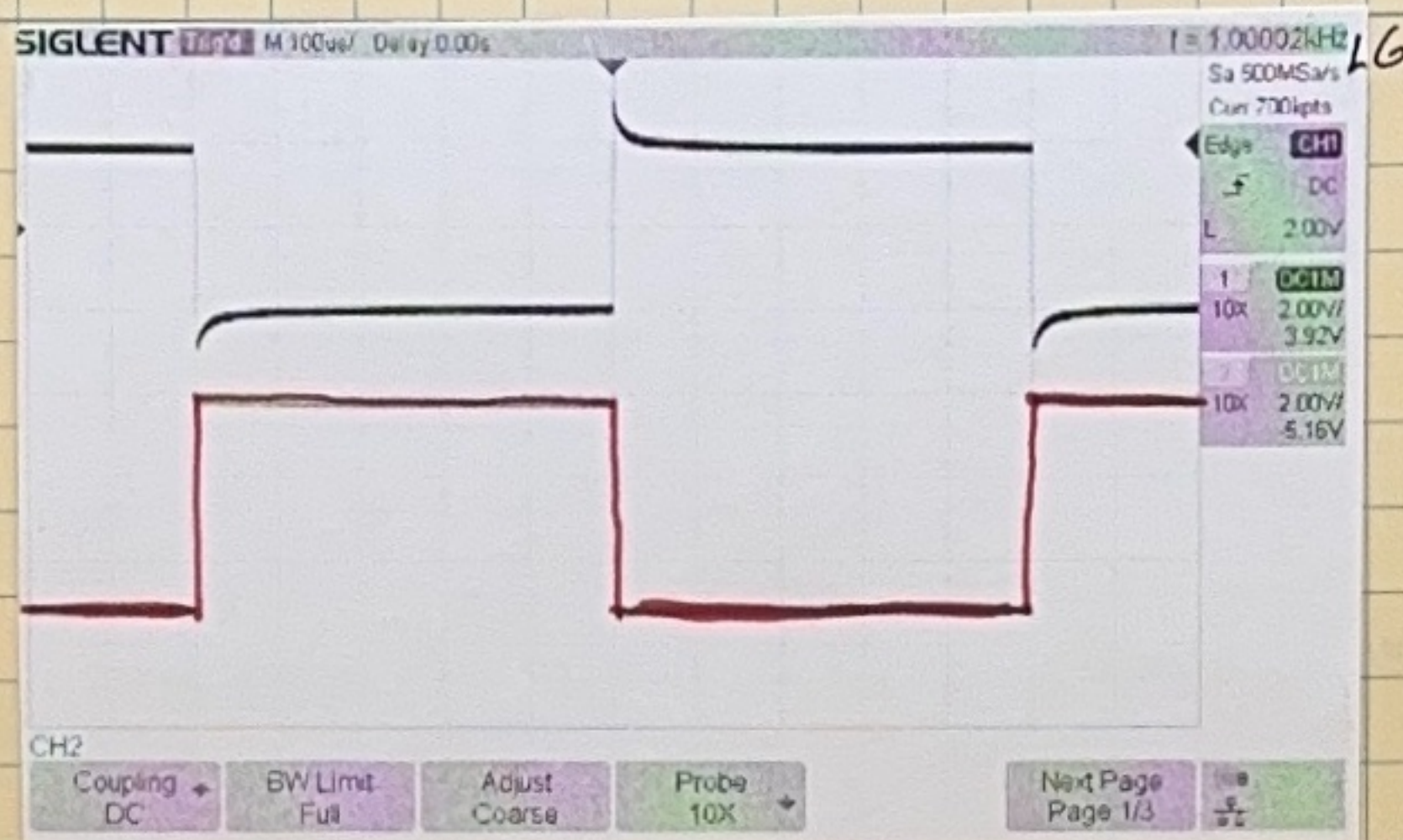


Figure 5: Speaker signal and port A output signal.

The Class D Amp inverts the output and there is small amounts of charge and discharge in the spikes on CH1 (Black waveform).

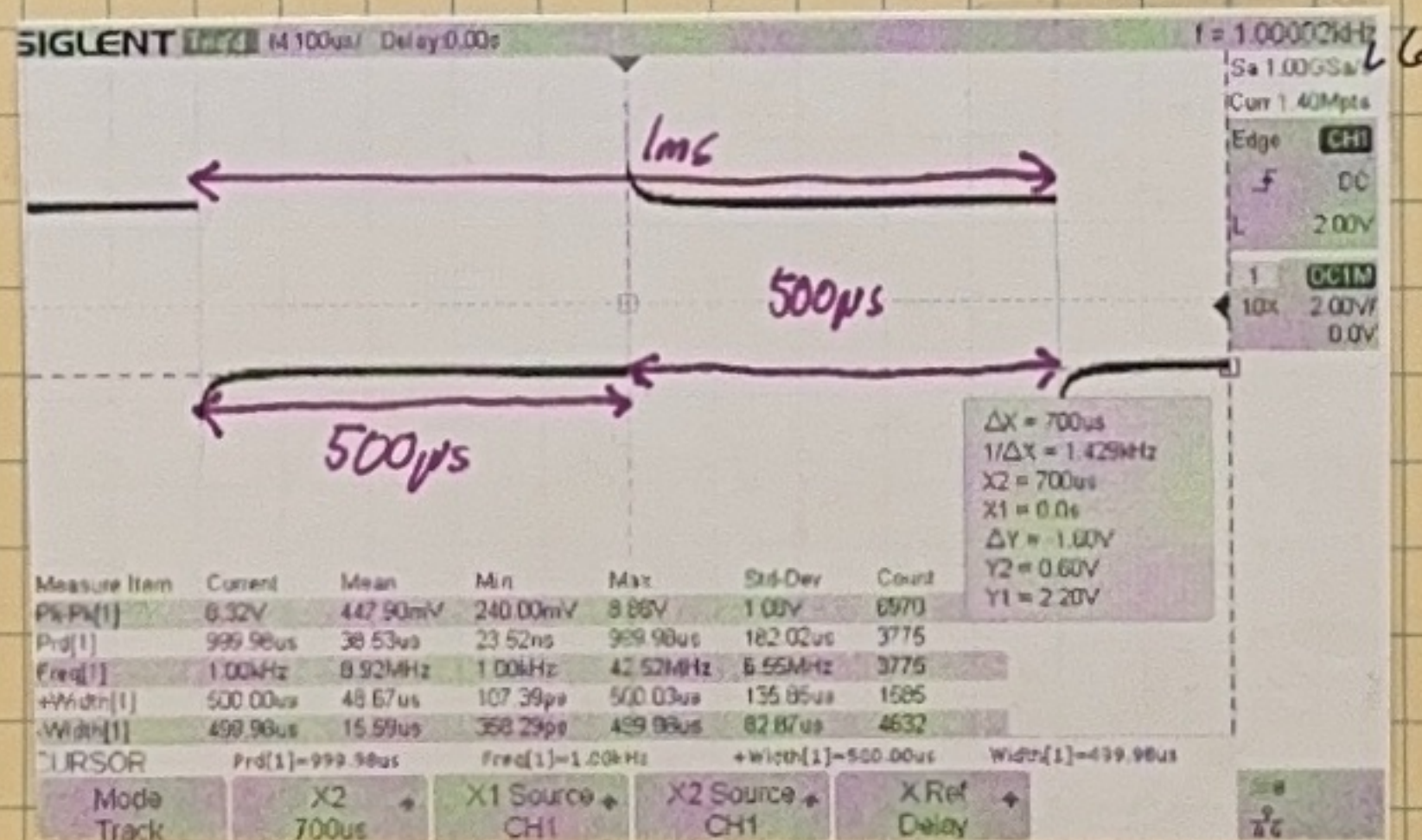


Figure 6: Measured frequency & PW, PS, Period, Duty Cycle %

Measurements taken with oscilloscope to find PW, PS, Period, frequency and Duty Cycle. Measured vs Predicted table 2 is below showing timing calculations aren't far off.

Quantity	Predicted at 1 us/TCY	Oscilloscope
PW	500ps	500ps
PS	500ps	499.98ps
Period	1ms	999.98ps
Frequency	1kHz	1.00002 kHz
Duty cycle	50%	49.999 %

Table 2: Predicted @ 1ps per TCY vs Measured values

Almost all values are well within 1% tolerance of predicted. small timing difference can be due to the crystal or cap values.

9/14/2026
LR6

Lab 4 - Funky Muzak

49

Setup:

; add device setup here

; Bank 0

BCF 0x03,6 ;Set Bank 0

BCF 0x03,5 ;Set Bank 0

CLRF 0x06 ;Setting PortB on reset low

CLRF 0x05 ;Setting PortA on reset low

; Bank 1

BCF 0x03,6 ;Set Bank 1

BSF 0x03,5 ;Set Bank 1

MOVLW 0x00

MOVWF 0x85 ;TRISA set PortA as outputs

MOVLW 0x01

MOVWF 0x86 ;TRISB set RB0 as input

; Bank 2

BSF 0x03,6 ;Set Bank 2

BCF 0x03,5 ;Set Bank 2

BSF 0x09,1 ;Setting Bit 1 High for CM2CON1

; Bank 3

BSF 0x03,6 ;Set Bank 3

BSF 0x03,5 ;Set Bank 3

CLRF 0x88 ;Setting the I/O to digital ANSEL PortA

CLRF 0x89 ;Setting the I/O to digital ANSELH PortB

;Reset to Bank 0

BCF 0x03,6 ;Set Bank 0

BCF 0x03,5 ;Set Bank 0

LG

Figure 7: SFR's for setup



From freq counter:

LG PW = 499.9938 μ s

PS = 500.0039 μ s

Period = 999.998 μ s

Figure 8: Measured output for speaker proving 1kHz signal

Evidence:

- circuit schematic and Class D interface
- loading/electrical analysis
- complete flowchart set
- final source code, fully include or referenced
- nominal timing calculations
- expected timing using the previously measured instruction cycle time.
- oscilloscope capture and measurements
- frequency counter measurement
- comparison of predicted and measured behavior
- evidence of no active button forces a output low.

LG

9/14/2026

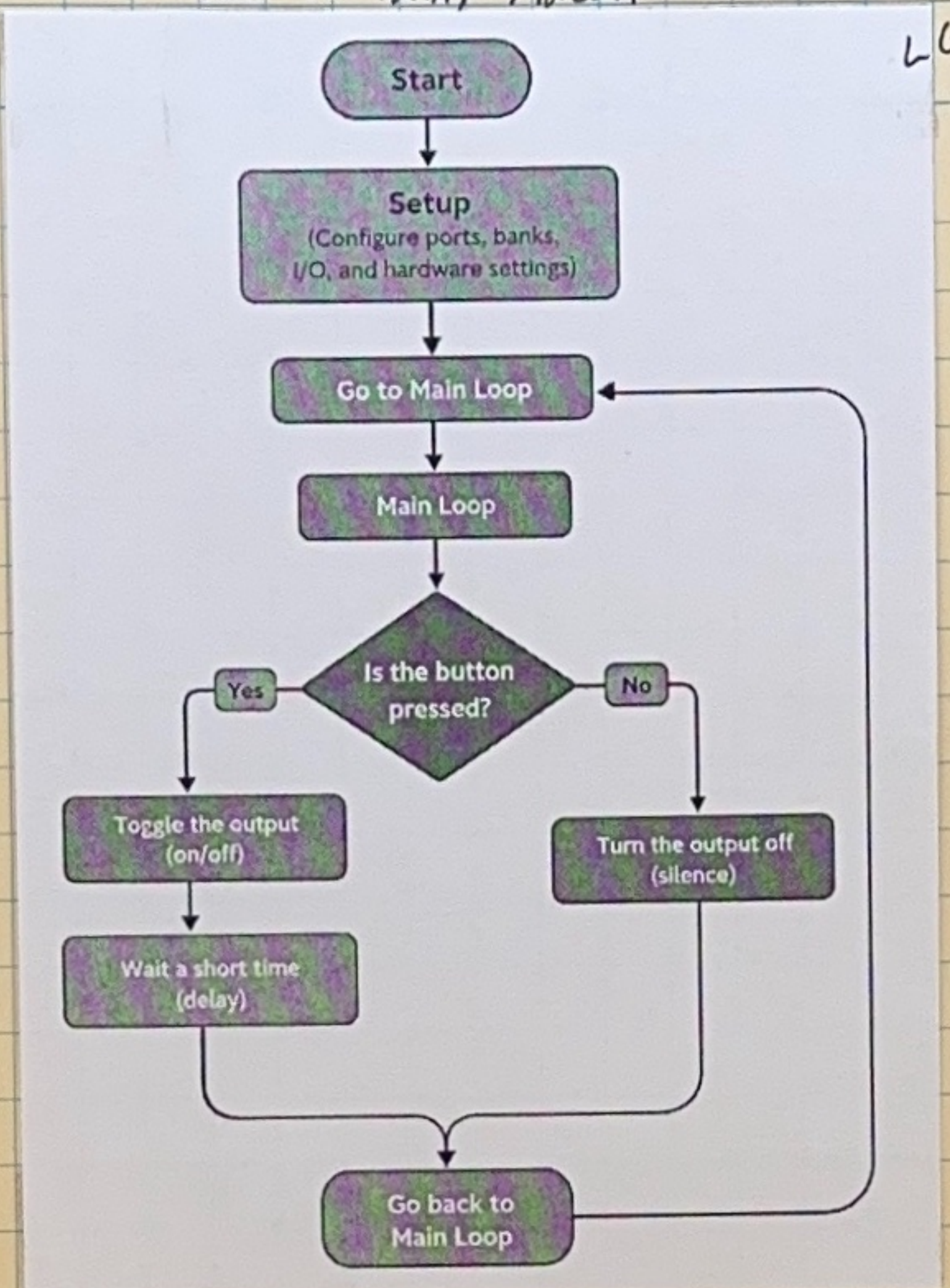


Figure 9: Flow chart of Setup and Main code

Part 2 - Select Notes with Dip Switches and look up tableGoal:

- Rewrite code^{LG} the priority switch scanning work from earlier PIC lab to select musical notes, retrieve the associated hexadecimal delay value from a program-memory look up table, and generate each selected note with one generalized delay mechanism.

- Note selection: use the eight DIP switches to select the first eight notes.

Switch	Note index	Note	Frequency (Hz)	Delay value (Hex)
RB0	1	C3	130.81	EE
RB1	2	D3	146.83	D4
RB2	3	E3	164.81	BD
RB3	4	F3	174.61	B2
RB4	5	G3	196.00	9F
RB5	6	A3	220.00	8E
RB6	7	B3	246.94	7E
RB7	8	C4	261.63	77

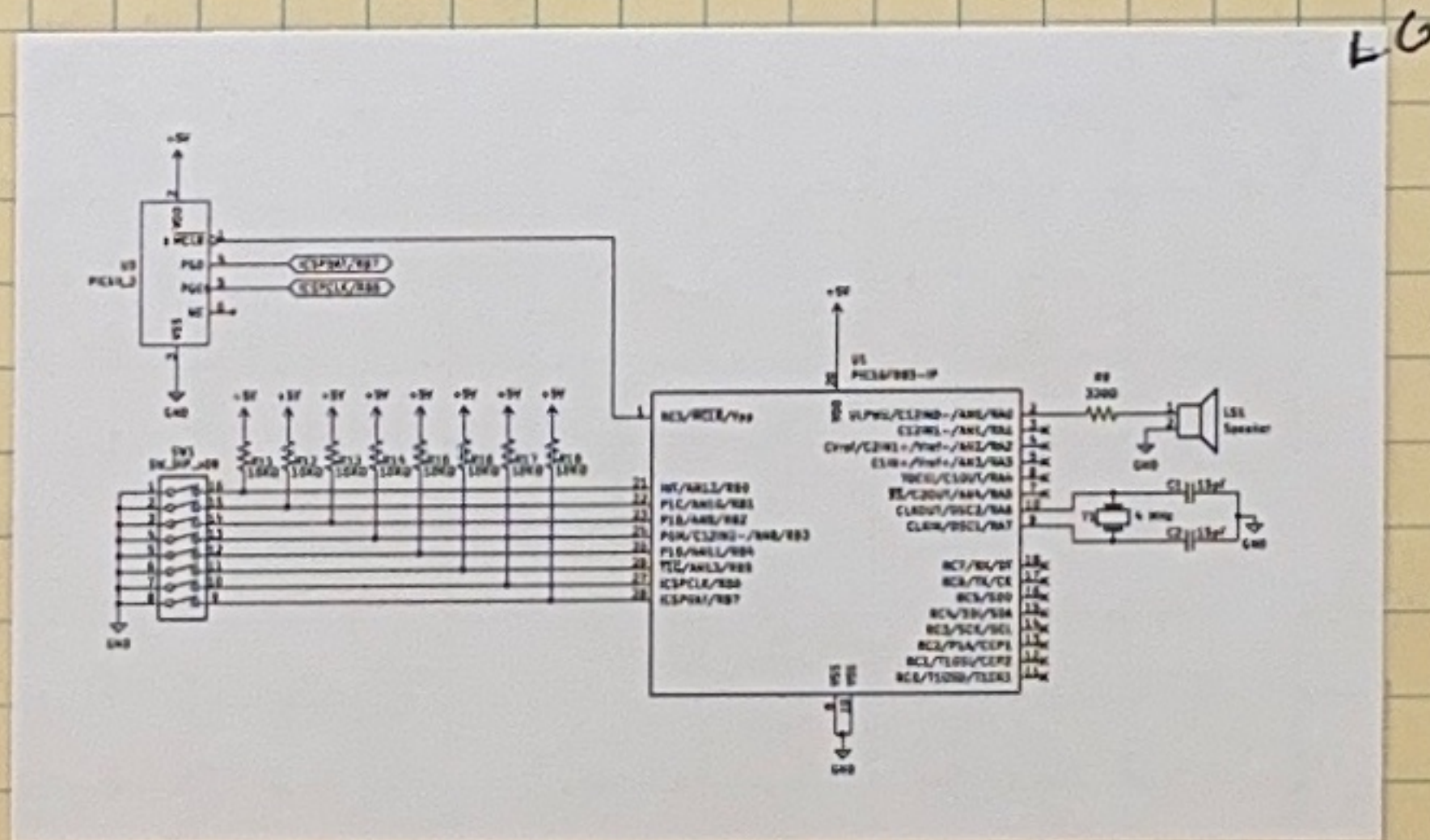
-^{LG}Table 3: RB portB connections for Notes of the look up table

- If no switch is active, the normalized note selection value must be 0 and the tone output must remain low. If multiple switches are active, the highest musical note has priority. The highest note index wins.

Before Lab:

- Reuse and adapt the priority-switch scanning algorithm develop previously. The input-scanning portion should determine the active note and store the normalized note index in a general purpose register.

Create a program-memory look-up table that was the note index to retrieve the provided hexadecimal delay values. A computed look-up using PCL and RETLW is appropriate. Research and document any PIC program-counter behavior or SFR information required to use the selected method correctly. The look up process should update a delay-count value used by the same generalized half cycle delay mechanism for every note. Do not modify the provided hexadecimal values. Do not create a different delay algorithm for each note. Before implementing note specific timing corrections, stop and examine the table and your generalized delay. The intended solution does not require individually tuning eight different frequencies.



LookUp: LG
 MOVF indexcount, W
 ADDWF PCL, F - jump into table below

```

RETLW 0x00      ;index 0 Silent
RETLW 0xEE      ;index 1 C3
RETLW 0xD4      ;index 2 D3
RETLW 0xBD      ;index 3 E3
RETLW 0xB2      ;index 4 F3
RETLW 0x9F      ;index 5 G3
RETLW 0x8E      ;index 6 A3
RETLW 0x7E      ;index 7 B3
RETLW 0x77      ;index 8 C4
  
```

Delay:

OutLoop:

MOVLW 0x04 1Tcy

MOVWF incount 1Tcy

InLoop:

DECFSZ incount, 1

GOTO InLoop

DECFSZ outcount, 1

GOTO OutLoop

RETURN 2Tcy

• The lookup table reserves a value from the test/set code and then comes here to grab it's desired value to play the note as selected.

• The delay section works with inloop and outloop because of the frequency needed to reach @ 1Tcy = 1µs due to the frequency of 170.81KHz = hex value of (EE) half period = 3823µs which EE's decimal value of 238 which makes $3823\mu s / 238 = 16\mu s$ per count. The value of 4 moved into incount makes that 16µs cycles per counts. which gives it so no matter what value is selected we can just multiple by 4 to give desired output.

• Figure 11: Lookup table code and Delay code

Main:

; main code goes here

CALL SelectSwitch ;Determine which switch is active 2Tcy

MOVF indexcount, W ;Check if no switch is active 1Tcy

BTFSC 0x05, 2 ;Check and make RA0 silent 1(2)Tcy N(2+1)-1

GOTO Silent 2Tcy

CALL LookUp 2Tcy; Call to the Look up table and check which index is selected

MOVWF outcount 1Tcy; Move the value to outcount for delay to play note

CALL Delay 2Tcy; Call delay and play frequency for note selected

MOVLW 0x01 1Tcy; Toggle RA0

XORWF 0x05, 1 1Tcy; Toggle TA0

GOTO Main 2Tcy

Silent:

BCF 0x05, 0 1Tcy; Force RA0 low - silence

GOTO Main 3Tcy

• The call sections will be on pg 53. They show how it selects which input is being read and the takes that value and decides in it goes to silent or if it plays a note.

• Figure 12: Main codes operation

Main Tcy = 15µs

LG

SelectSwitch:

```

BTFSS 0x06,7 ;Skip next line if RB7 is set 1(2)Tcy
GOTO Test6   ;If not Test the next switch 2Tcy
GOTO Set8     ;If RB7 is high got to Set8 2Tcy

```

LG
 Last seen
 Tcy = 4µs

Test6:

```

BTFSS 0x06,6 ;Skip next line if RB6 is set
GOTO Test5   ;If not Test the next switch
GOTO Set7     ;If RB6 is high got to Set7

```

Test5:

```

BTFSS 0x06,5 ;Skip next line if RB5 is set
GOTO Test4   ;If not Test the next switch
GOTO Set6     ;If RB5 is high got to Set6

```

Most seen
 Tcy = 24µs

Figure 13: Test to see which switch is selected

The purpose is to see which input on PORTB is selected and take that value into either it's set note selection or ~~the~~ keep testing to see what is selected. Lets say RB7 is selected it goes to set 8 see Figure 14 for that code.

Set8:

```

MOVLW 0x08 ;Move W into indexcount 1Tcy
MOVWF indexcount ;Send W to indexcount for index table 1Tcy
RETURN ;Return the indexcount for index table 2Tcy

```

Most seen
 Tcy = 4µs

Set7:

```

MOVLW 0x07 ;Move W into indexcount
MOVWF indexcount ;Send W to indexcount for index table
RETURN ;Return the indexcount for index table

```

Set6:

```

MOVLW 0x06 ;Move W into indexcount
MOVWF indexcount ;Send W to indexcount for index table
RETURN ;Return the indexcount for index table

```

Figure 14: Set index count value for lookup table

Here it just selects it index value and moves it it^{LG} to the indexcount to then be used to select its Hex value used in the delay code as seen in code on figure 11 pg 52.

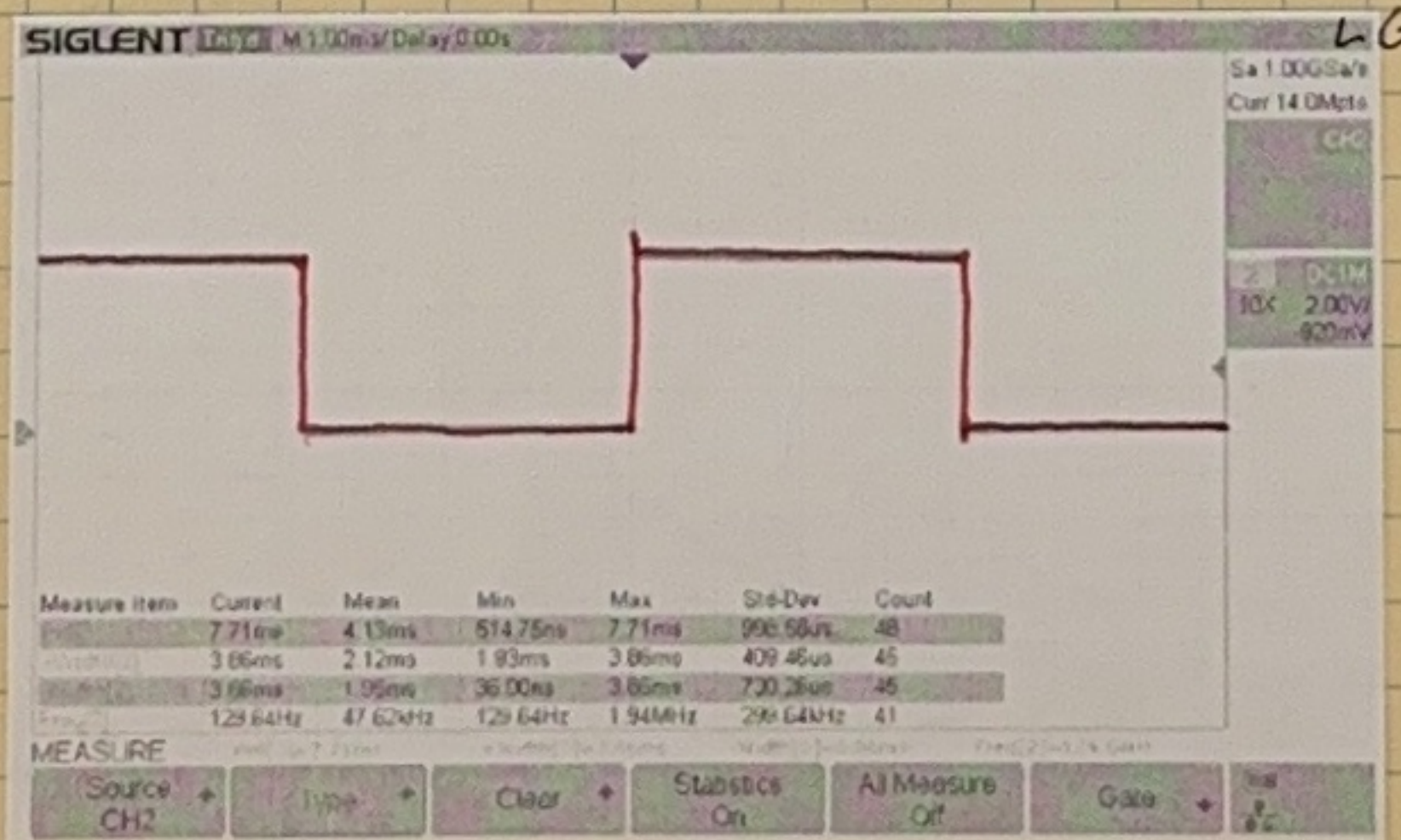
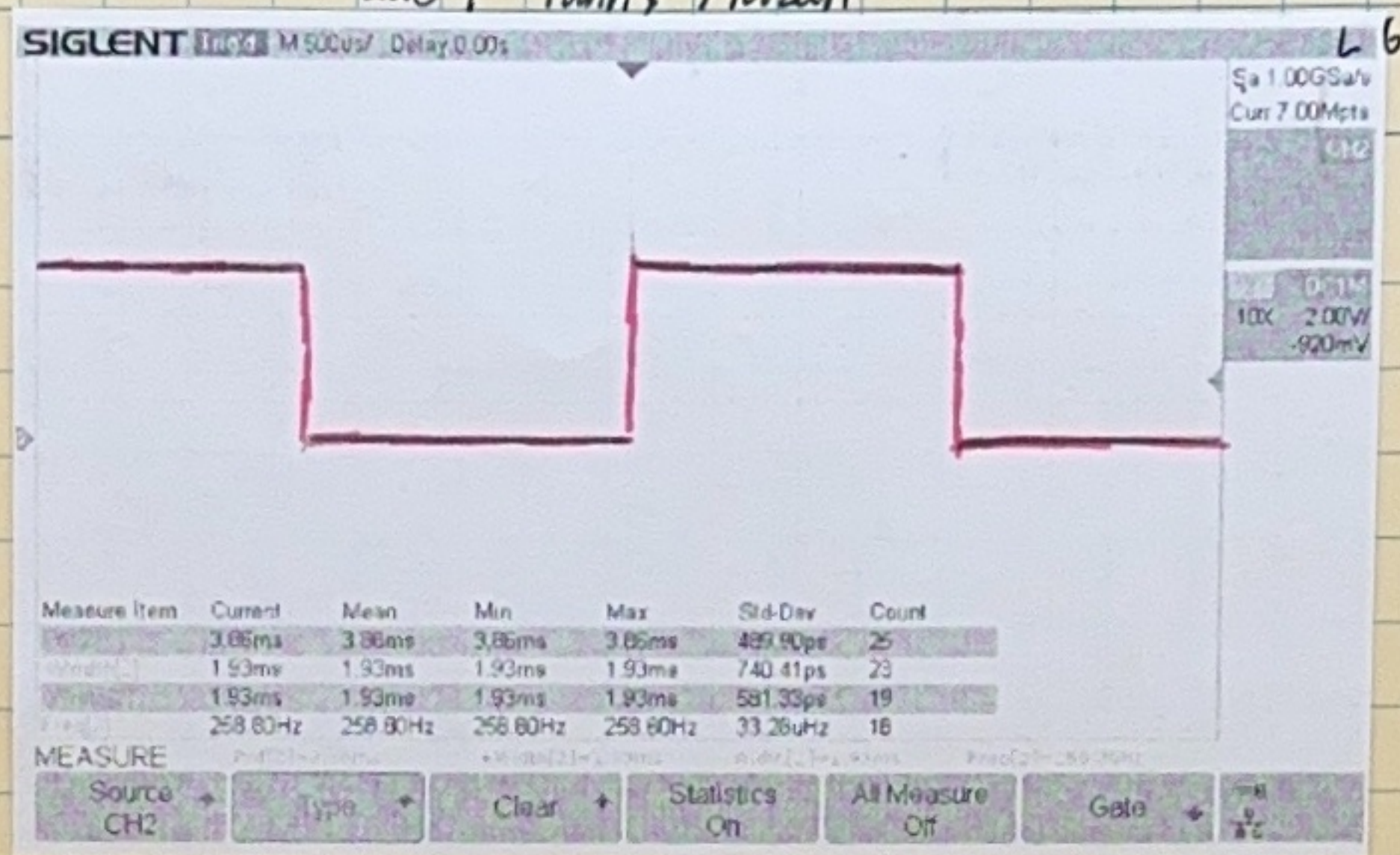


Figure 15: RBD set freq of 130.81 desired/measured freq = 129.64 capture

The measured Figure shows the least significant note of C3.



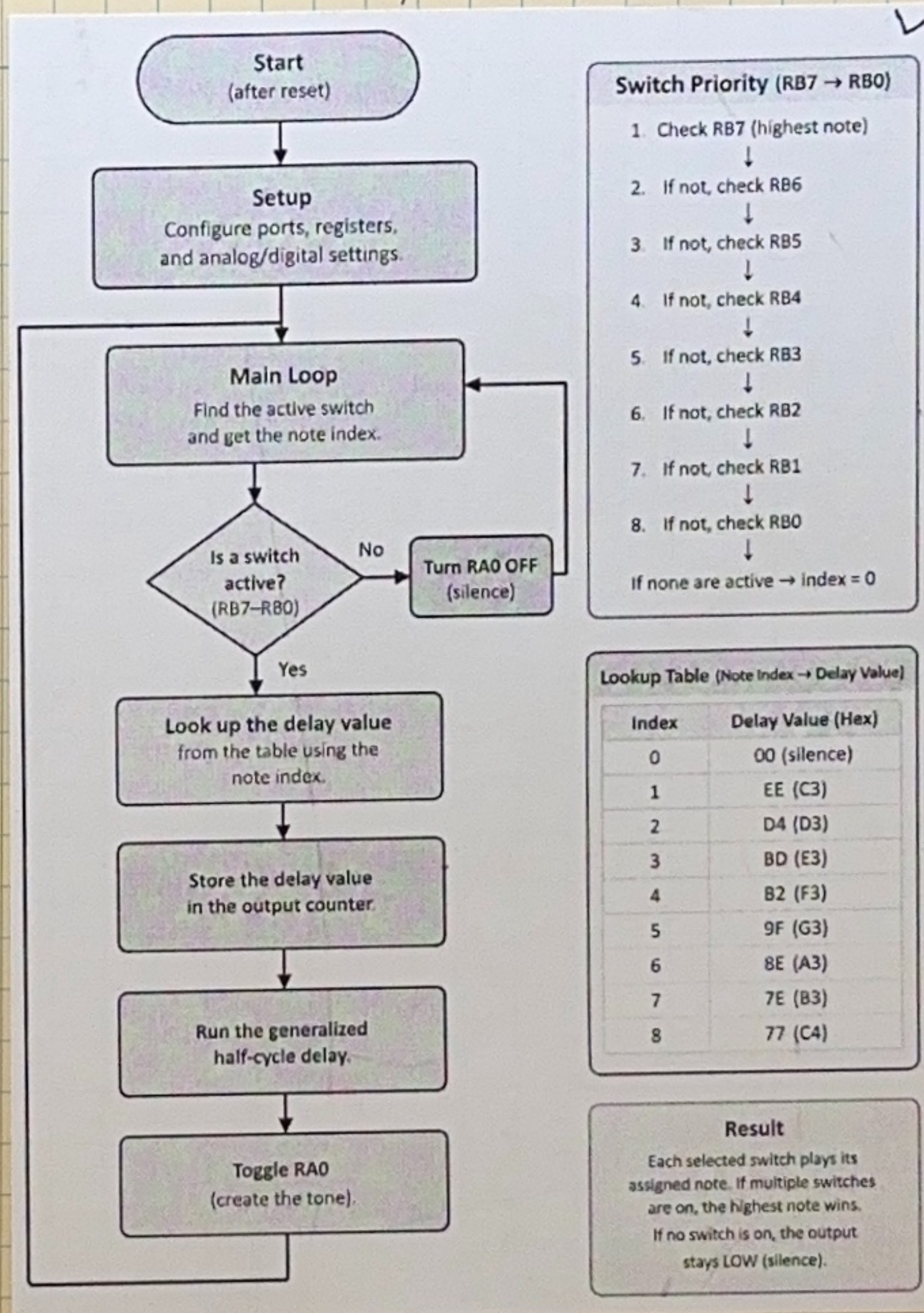
• Figure 16: RB7 set 261.63Hz desired frequency / Measured freq 258.8Hz capture
 - The measured waveform is of what frequency is set for index table Look up table that is note C4 which its desired frequency was 261.63Hz

Quantity	Predicted at 1 us/TCY		Oscilloscope		Frequency Counter	
	C3	C4	C3	C4	C3	C4
PW	3.822ms	1.911ms	3.86ms	1.93ms	3.85649ms	1.9319ms
PS	3.833ms	1.911ms	3.86ms	1.93ms	3.85646ms	1.9319ms
Period	7.645ms	3.822ms	7.71ms	3.85ms	7.7134ms	3.8639ms
Frequency	130.81 Hz	261.63Hz	129.64Hz	258.8Hz	129.6347Hz	258.7997Hz
Duty cycle	50%	50%	50%	50%	50%	50%

• Table 4: Measured vs Predicted for notes C3 & C4 using both ways to measure
 - The oscilloscope's resolution can't see very far to precise measurements but it can still provide valid measurements.

Switch	Note index	Note	Frequency (Hz)	Measured frequency
RB0	1	C3	130.81	129.637 Hz
RB1	2	D3	146.83	145.436 Hz
RB2	3	E3	164.81	163.029 Hz
RB3	4	F3	174.61	173.134 Hz
RB4	5	G3	196.00	193.727 Hz
RB5	6	A3	220.00	216.83 Hz
RB6	7	B3	246.94	244.265 Hz
RB7	8	C4	261.63	258.805 Hz

• Table 5: Measured vs desired frequencies



• Figure 17: Flow Chart for Part 2 code

- all calculations for loading are in part 1 pg 47. The flowchart shows the priority order and index values for the delay loop. It shows how it remains silent as when no switches are selected.

TJR
9-16-24

L6

Part 3- Build the 16 Key Muzak Keyboard:Goal:

- Replace the DIP-switch input with the previously developed 4x4 Keypad scanner while preserving the reusable note-selection, look-up, delay, and output structure.

- The purpose of this part is integration. Keypad scanning itself has already been developed and verified in an earlier lab and does not require a separate instructor checkoff here.

Keypad note assignment:

- Assign notes in ascending musical order following the physical adjacency of the Keypad:

1 2 3 A

4 5 6 B

7 8 9 C

* 0 # D

- Use this mapping:

Physical key	Note index	Note	Frequency (Hz)	Delay value (Hex)
1	1	C3	130.81	EE
2	2	D3	146.83	D4
3	3	E3	164.81	BD
A	4	F3	174.61	B2
4	5	G3	196.00	9F
5	6	A3	220.00	8E
6	7	B3	246.94	7E
B	8	C4	261.63	77
7	9	D4	293.66	6A
8	10	E4	329.63	5E
9	11	F4	349.23	59
C	12	G4	392.00	4F
*	13	A4	440.00	47
0	14	B4	493.88	3F
#	15	C5	523.25	3B
D	16	D5	587.33	35

Table 6: Index for lookup table for 4x4 Keypad

- The physical label printed on the key no longer determines priority. If multiple keys are detected, the highest assigned musical note must win. The keypad-scanning logic should therefore produce the highest active note index, not the numerically or alphabetically largest printed keypad character.

L6

Lab 4 - Funky Muzak

• Before Lab:

- Reuse the matrix keypad circuit, electrical analysis, scan-state work, and scanning algorithm developed previously. Reference unchanged lab book pages and source instead of recreating them. Adapt the keypad result so that the input-handling portion of the program stores the normalized musical note index shown in table 6.

- The rest of the tone-generation path should continue to operation from that normalized value:
physical keypad
↓

Normalized note selection
provided hexadecimal delay value
↓
generalized half-cycle delay
↓
tone output

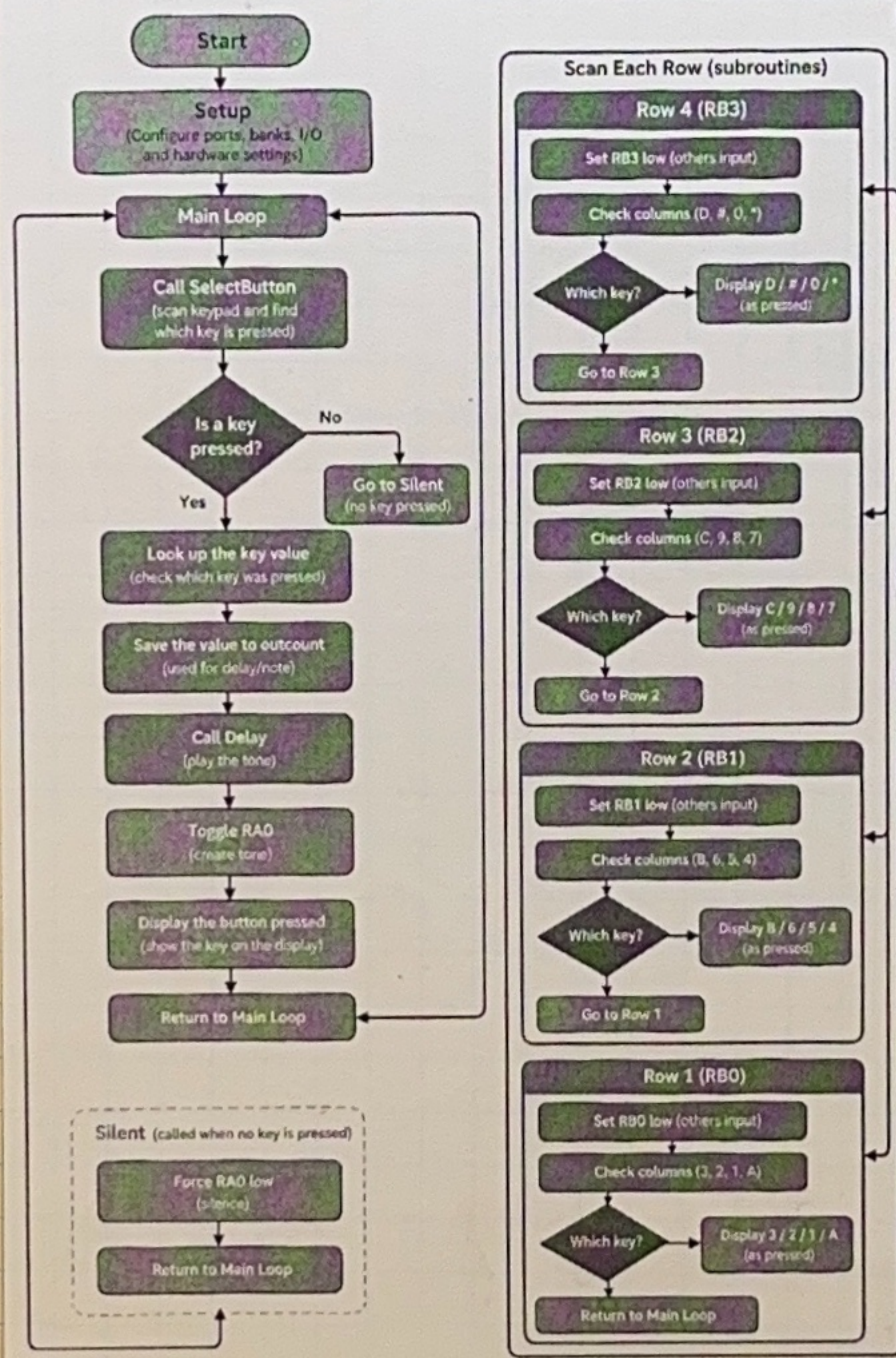
• In the Lab:

- 1) Connect the keypad using the previously verified interface
- 2) Confirm that no key is pressed forces⁶ the tone output low.
- 3) Verify each of the 16 keys individually.
- 4) Confirm that physically adjacent keys produce the expected ascending note sequences shown in the assignment.
- 5) Verify multiple-key behavior ^{L6} only after the electrical current paths are known to be safe.
- 6) Confirm that the highest active musical note has priority.
- 7) Measure at least one low note and one high note with the oscilloscope and frequency counter.
- 8) Observe the output while the keypad is being scanned. Determine how input polling, selection logic, look-up activity, branching, and subroutine overhead affect the half cycle timing.
- 9) Compare the measured frequencies with the target values and with the behavior of the generalized delay implementation.
- 10) Document troubleshooting and integration changes.

L6

```
; config statements should precede project file includes.
#include <xc.inc>
indexcount EQU 0x20 ;use data memory address 0x20 as an 8-bit loop counter
outcount EQU 0x21 ;use data memory address 0x21 as an 8-bit loop counter
incount EQU 0x22 ;use data memory address 0x22 as an 8-bit loop counter
highcount EQU 0x23 ;for memory address 0x23 to hold values of columns/rows
lowcount EQU 0x24 ;for memory address 0x24 to hold values of columns/rows
```

• Figure 18: Included EQU's for memory data in code.



The call Select button goes to scanning and testing rows.

Figure 19: Flow Chart of code and its functions with hardware

The scan rows and columns code see Lab 2 - Part 4 pg 23 for similar scanning the difference is in the test and display, as the display is included for fun.

TestD:
BTFSS highcount, 5 ;Test if D is pressed
GOTO TestHash ;GOTO Test for #
GOTO DisplayD ;GOTO the Display for D

TestHash:
BTFSS highcount, 7 ;Test if # is pressed
GOTO Test0 ;GOTO Test for *
GOTO DisplayHash;GOTO the Display for #

Test0:
BTFSS lowcount, 0 ;Test if 0 is pressed
GOTO TestStar ;GOTO the Display for *
GOTO Display0 ;GOTO the Display for 0

TestStar:
BTFSS highcount, 6 ;Test if * is pressed
GOTO TestC ;GOTO Test for C
GOTO DisplayStar;GOTO the Display for *

Figure 20: Or Test priority code
This code is the priority of the which button/key has the highest priority which is the highest note.

```

DisplayDal:
    MOVLW 0x80
    MOVWF indexcount
    MOVLW 0x24      ;Select the character set for display 0 or whatever
    MOVWF 0x07      ;Send W to PortC
    RETURN

DisplayHash:
    MOVLW 0x0E      ;Move W into indexcount
    MOVWF indexcount ;Send W to indexcount for index table
    MOVLW 0x23      ;Select the character set for display #
    MOVWF 0x07      ;Send W to PortC
    RETURN

DisplayStar:
    MOVLW 0x0C      ;Move W into indexcount
    MOVWF indexcount ;Send W to indexcount for index table
    MOVLW 0x2A      ;Select the character set for display *
    MOVWF 0x07      ;Send W to PortC
    RETURN

DisplayD:
    MOVLW 0x0F      ;Move W into indexcount
    MOVWF indexcount ;Send W to indexcount for index table
    MOVLW 0x44      ;Select the character set for display D
    MOVWF 0x07      ;Send W to PortC
    RETURN
    
```

for the display code
all that was added
was the index count
value for the lookup table.

Figure 21: Display code with note value for lookup tables

```

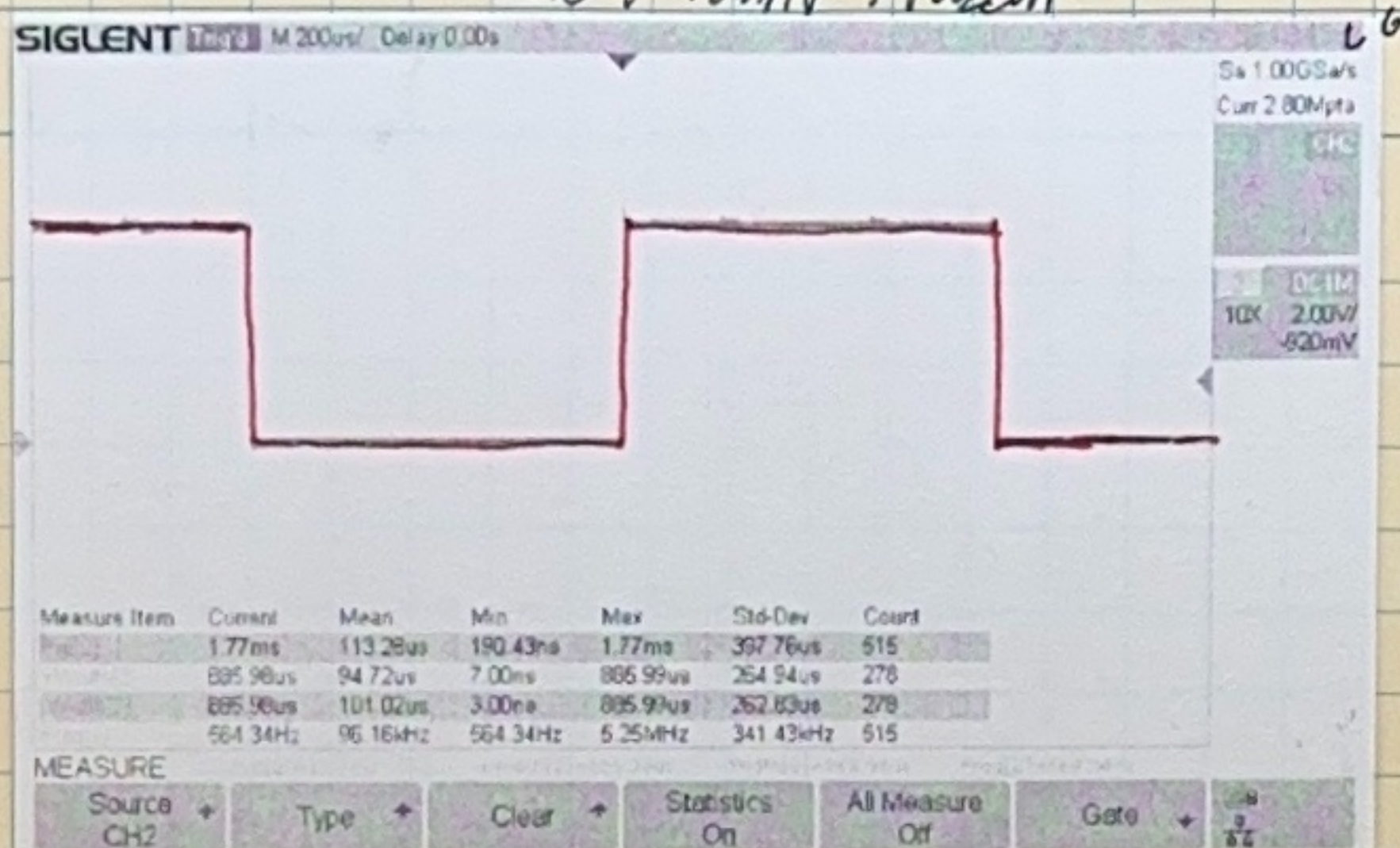
LookUp:
    MOVF indexcount, W
    ADDWF PCL, 1 ;ADDWF PCL, F - jump into table below
    RETLW 0xEE      ;index 1 C3
    RETLW 0xD4      ;index 2 D3
    RETLW 0xBD      ;index 3 E3
    RETLW 0xB2      ;index 4 F3
    RETLW 0x9F      ;index 5 G3
    RETLW 0x8E      ;index 6 A3
    RETLW 0x7E      ;index 7 B3
    RETLW 0x77      ;index 8 C4
    RETLW 0x6A      ;index 9 D4
    RETLW 0x5E      ;index 10 E4
    RETLW 0x59      ;index 11 F4
    RETLW 0x4F      ;index 12 G4
    RETLW 0x47      ;index 13 A4
    RETLW 0x3F      ;index 14 B4
    RETLW 0x3B      ;index 15 C5
    RETLW 0x35      ;index 16 D5

Delay:
    OutLoop:
        MOVLW 0x04
        MOVWF incount
    InLoop:
        DECFSZ incount, 1
        GOTO InLoop
        DECFSZ outcount, 1
        GOTO OutLoop
    RETURN
    
```

The lookup table has the
notes listed on¹⁶ in LSB to
MSB.

Figure 22: Lookup table with delay

LG

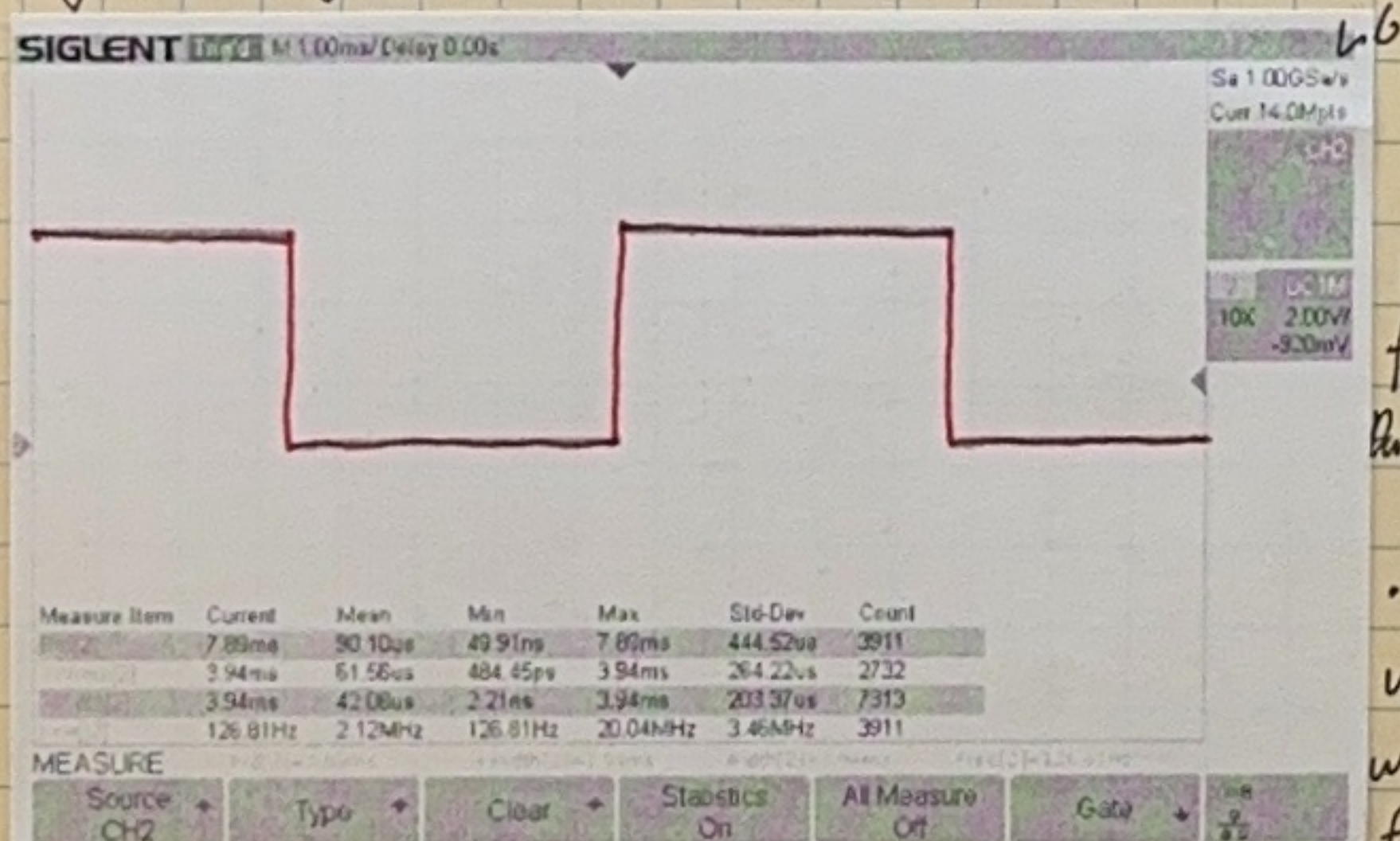


Oscope

frequency counter

PW: 937.99 μ s 438.605 μ s
 PS: 437.98 μ s 437.94 μ s
 Period: 1.88ms 1.8754 μ s
 Frequency: 564.34Hz 584.04Hz
 Duty Cycle: 50% 50%

Figure 23: Highest note selected note waveform



oscope

frequency counter

PW: 3.94ms 3.943ms
 PS: 3.94ms 3.943ms
 Period: 7.89ms 7.885ms
 Frequency: 126.804Hz 126.967Hz
 Duty Cycle: 50% 50%

• These are the measured values for the wave forms with both the oscilloscope and frequency counter.

Figure 24: Lowest note selected waveform note

Setup:

```

; Bank 0
BCF 0x03,6 ;Set Bank 0
BCF 0x03,5 ;Set Bank 0
CLRF 0x05 ;Setting PortA on reset low
CLRF 0x06 ;Setting PortB on reset low known
CLRF 0x07 ;Set PortC on reset to LOW

; Bank 1
BCF 0x03,6 ;Set Bank 1
BSF 0x03,5 ;Set Bank 1
CLRF 0x0B ;Set INTCON low to remove interrupt flags
CLRF 0x87 ;Set PortC as an Output TRISC
MOVLW 0xF0 ;Make RB0-RB3 outputs for the rows and RB4-RB7 inputs for the columns
MOVWF 0x86 ;Setting PortB I/O from TRISB
BCF 0x81,7 ; Make PortB have weak pull-ups OPTION_REG
MOVLW 0x00
MOVWF 0x85 ;TRISA set PortA as outputs

; Bank 2
BSF 0x03,6 ;Set Bank 2
BCF 0x03,5 ;Set Bank 2
BSF 0x09,1 ;Setting Bit 1 High for CM2CON1

; Bank 3
BSF 0x03,6 ;Set Bank 3
BSF 0x03,5 ;Set Bank 3
CLRF 0x88 ;Setting the I/O to digital ANSEL PortA
CLRF 0x89 ;Setting the I/O to digital ANSEL PortB

GOTO Main
  
```

Figure 25: SFR for bank selection

• These are the SFRs used to set up the banks and code. Setting up the different I/O's and config for how they work.

9/17/2026
LR6

Lab 4 - Funky Much

61

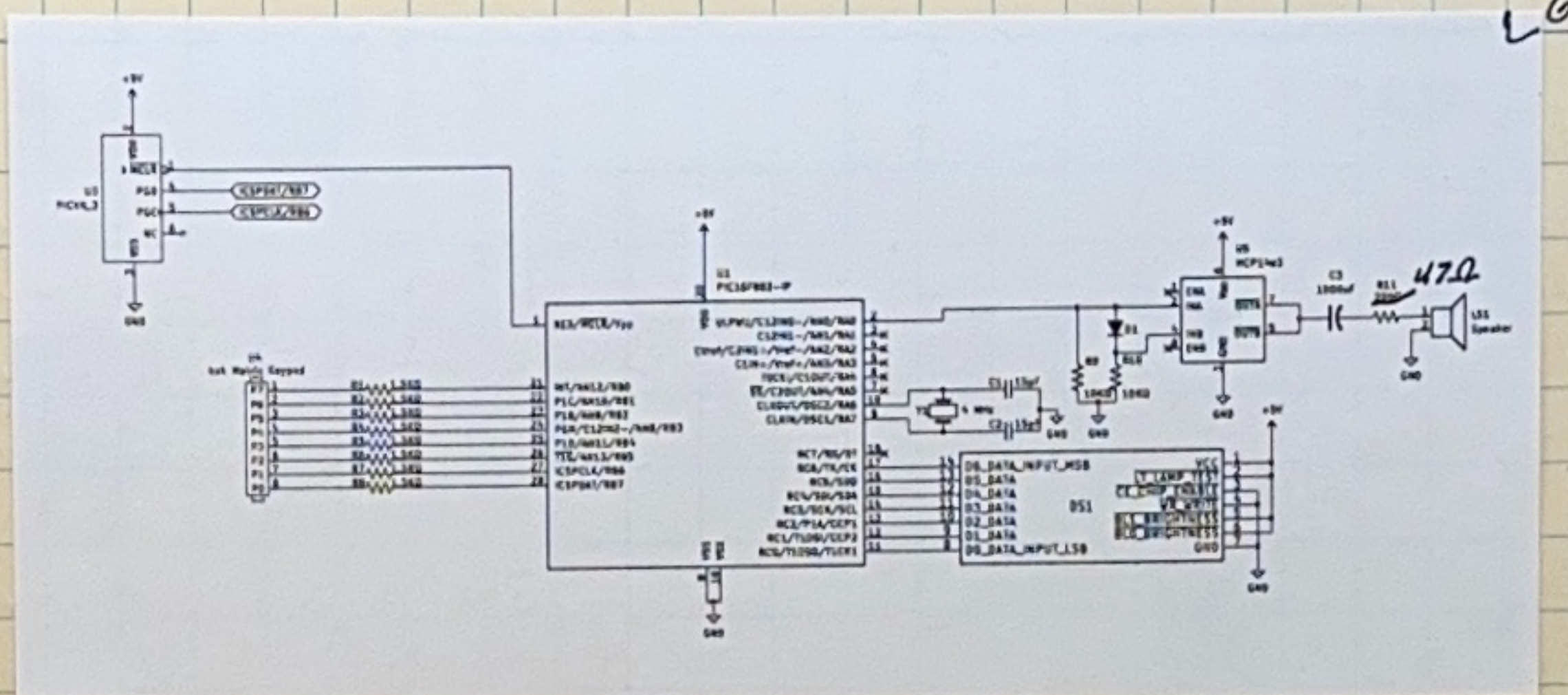


Figure 26: Full schematic with speaker and display.

Physical key	Note index	Note	Frequency (Hz)	Measured frequency
1	1	C3	130.81	126.809Hz
2	2	D3	146.83	141.887Hz
3	3	E3	164.81	158.582Hz
A	4	F3	174.61	168.127Hz
4	5	G3	196.00	187.480Hz
5	6	A3	220.00	209.034Hz
6	7	B3	246.94	234.416Hz
B	8	C4	261.63	247.775Hz
7	9	D4	293.66	276.707Hz
8	10	E4	329.63	310.180Hz
9	11	F4	349.23	327.017Hz
C	12	G4	392.00	366.039Hz
*	13	A4	440.00	404.866Hz
0	14	B4	493.88	452.907Hz
#	15	C5	523.25	482.164Hz
D	16	D5	587.33	533.659Hz

Table 7: Measured vs Wanted frequencies.

- So due to reused code and including the display code and the way each row and column is scanned in the code is 60Tcy before any tests are done so this is way as the frequency climbs the difference becomes greater and greater but this being reused code from both part 2 of this lab and Lab 2 part 4. So it was easy to get put together but to get this spot on each hex value would have to change the note. In this lab I was given the hex value so it diff⁶ didn't seem like each thing needed to get

9/17/2026
LR6

-Conclusion:

- Funky Muzak's Lab is learning and programming code to generate a tone and getting more and more keys to create different musical notes. The new thing used in this section was lookup table. This is where specific notes were stored and it is how the notes were then played using reusable code from Part 2. The final section is where the reused code that works was super helpful but it had limitations on timing. These note selections had a value that were used in a delay count that g^{LG} created the PW and PS for the tone to be played. The notes were given and with that the delay was created to best recreate that note. The first part was just using a button press to generate a 1KHz tone. The next was using a dip switch and selecting notes and proving priority. Part 3 was using the 4x4 matrix to play keys. The mastery was using the 7 segment as well but I left the code from part 4 of Lab 2 and it did mess with times and frequencies. Without that extra ^{LG} extra code it would bring the error % at the highest note played from 8.4% to 5.3%. Which is very large. These can get smaller based on what is desired but for this lab it is more about understanding reusable code and taking information and applying it.

TJR
9-17-26